AIM2 Preparation

Week 2 - Day 1: NumPy Fundamentals

5-Minute Review Notes | Arrays, Vectorization & Aggregation

Day 1 Goal

Build a foundation for numerical computing with NumPy: create arrays, inspect their structure, select data, perform fast element-wise calculations, summarize values, and reshape multidimensional data.

1. Import NumPy

```
import numpy as np
```

np is the conventional alias used for NumPy.

2. NumPy Arrays

```
scores = np.array([85, 75, 91, 65])

marks = np.array([[85, 92, 88],
[75, 80, 70],
[91, 95, 94]])
```

A NumPy array (ndarray) stores numerical data efficiently. A 1D array behaves like a numerical sequence; a 2D array represents rows and columns.

3. Inspecting an Array

```
scores.ndim
scores.shape
scores.size
scores.dtype
```

Property	Meaning
ndim	Number of dimensions
shape	Size of each dimension
size	Total number of elements
dtype	Data type of the elements

4. Indexing and Slicing

```
scores[0] # first value
scores[-1] # last value
scores[1:3] # slice

marks[0, 1] # row 0, column 1
marks[:, 0] # all rows, first column
```

Indexing selects individual positions; slicing selects ranges or sections of an array.

5. Creating Arrays Quickly

```
np.zeros(5)
np.zeros((3, 4))
np.ones(4)
np.arange(1, 10)
np.arange(0, 21, 2)
```

- zeros() creates arrays filled with 0.
- ones() creates arrays filled with 1.
- arange(start, stop, step) creates regularly spaced values; stop is excluded.

6. Vectorized Operations

```
arithMath = np.array([70, 80, 90, 100])

arithMath + 5
arithMath - 5
arithMath * 2
arithMath / 2
```

NumPy applies operations element by element without requiring a Python for-loop. This is called vectorization.

7. Element-wise Array Operations

```
a = np.array([10, 20, 30])
b = np.array([1, 2, 3])

a + b
a - b
a * b
a / b
```

With compatible shapes, arithmetic between arrays is performed element by element.

8. Broadcasting

```
scores + 5
```

Broadcasting allows NumPy to apply a smaller compatible value/array across a larger array. Here the scalar 5 is applied to every element.

9. Aggregation Functions

```
scores.sum()
scores.mean()
scores.min()
scores.max()
```

Aggregations reduce many values into useful summaries such as total, average, minimum, and maximum.

10. axis=0 and axis=1

```
marks.mean(axis=0)
marks.mean(axis=1)
```

axis=0 calculates down the rows for each column. **axis=1** calculates across the columns for each row.

11. Boolean Filtering

```
scores > 80
scores[scores > 80]
```

The comparison creates a Boolean mask. Using that mask inside the array returns only values where the condition is True.

12. Reshape and Transpose

```
arr = np.arange(1, 7)
arr.reshape(2, 3)

marks.T
```

reshape() changes the dimensions while preserving the same number of elements. .T transposes rows and columns.

Important Reshape Rule

The total number of elements must stay the same. For example, 6 values can reshape to (2,3) or (3,2), but not (4,2).

Day 1 Practice Highlights

- Created 1D and 2D arrays.
- Inspected ndim, shape, size and dtype.
- Practiced indexing and slicing.
- Created zeros, ones and number sequences with arange().
- Performed vectorized arithmetic and broadcasting.
- Calculated mean, min, max and other summaries.
- Practiced Boolean filtering.
- Reshaped arrays and used transpose.

Day 1 Mini-Project - Student Score Analyzer (NumPy Edition)

- Stored student scores in NumPy arrays.
- Calculated numerical summaries such as average, highest and lowest scores.
- Used array operations instead of manual loops where appropriate.
- Practiced selecting and transforming numerical data.

Quick Syntax Cheat Sheet

```
np.array(...)
arr.ndim
arr.shape
arr.size
arr.dtype
arr[index]
arr[start:stop]
np.zeros(...)
np.ones(...)
np.arange(start, stop, step)
arr + value
arr * value
arr.mean()
arr.min()
arr.max()
arr[condition]
arr.reshape(rows, cols)
arr.T
```

Key Takeaways

- NumPy arrays are optimized for numerical data.
- Vectorization performs operations on entire arrays without explicit Python loops.
- Broadcasting applies compatible smaller values/arrays across larger arrays.
- Boolean masks filter numerical data efficiently.
- shape describes dimensions; size counts all elements.
- axis=0 works column-wise; axis=1 works row-wise.
- reshape changes structure, not the number of elements.

- NumPy is the numerical foundation used underneath much of the Python data/ML ecosystem.

Progress

Week 2 Day 1 - NumPy Fundamentals: COMPLETE
Next completed topic: Week 2 Day 2 - Pandas Fundamentals.